Day 04

Advanced Python Programming Concepts

Internship Task Documentation

Topic	Advanced Python Programming Concepts
Prepared by	Sakshi Ramesh Karale
Date	17 October 2026

1. Store and process student or employee records.

Program:

```
students = [
   {"id": 101, "name": "Rahul", "marks": 85},
   {"id": 102, "name": "Priya", "marks": 90},
   {"id": 103, "name": "Amit", "marks": 75}
]


for student in students:
   print(student)


total = sum(student["marks"] for student in students)
print("Total Marks:", total)
print("Average Marks:", total / len(students))
```

Output:

```
{'id': 101, 'name': 'Rahul', 'marks': 85}
{'id': 102, 'name': 'Priya', 'marks': 90}
{'id': 103, 'name': 'Amit', 'marks': 75}
Total Marks: 250
Average Marks: 83.33333333333333
```

2. Search for a record by ID or name.

Program:

```
students = [
   {"id": 101, "name": "Rahul", "marks": 85},
   {"id": 102, "name": "Priya", "marks": 90},
   {"id": 103, "name": "Amit", "marks": 75}
]
```

```
search = input("Enter Student ID or Name: ")

found = False

for student in students:
    if (str(student["id"]) == search or
            student["name"].lower() == search.lower()):
        print("Record Found:", student)
        found = True
        break

if not found:
    print("Record not found.")
```

Output:

Enter Student ID or Name: Priya

Record Found: {'id': 102, 'name': 'Priya', 'marks': 90}

3. Sort records by name or marks.

Program:

```
students = [
    {"id": 101, "name": "Rahul", "marks": 85},
    {"id": 102, "name": "Priya", "marks": 90},
    {"id": 103, "name": "Amit", "marks": 75}
]

choice = input("Sort by name or marks: ").lower()
```

```
if choice == "name":
    students.sort(key=lambda s: s["name"])
elif choice == "marks":
    students.sort(key=lambda s: s["marks"], reverse=True)
else:
    print("Invalid choice!")

print("Sorted Records:")

for student in students:
    print(student)
```

Output:

```
Sort by name or marks: marks
Sorted Records:
{'id': 102, 'name': 'Priya', 'marks': 90}
{'id': 101, 'name': 'Rahul', 'marks': 85}
{'id': 103, 'name': 'Amit', 'marks': 75}
```

4. Handle invalid input using exception handling.

Program:

```
try:
    sid = int(input("Enter Student ID: "))
    marks = float(input("Enter Student Marks: "))

    print("Student ID:", sid)
    print("Student Marks:", marks)

except ValueError:
```

```
    print("Invalid input! Please enter numbers only.")
```

Output:

```
Enter Student ID: abc
Invalid input! Please enter numbers only.
```

5. Practice working with lists and dictionaries.

Program:

```
students = [
    {"id": 101, "name": "Rahul", "marks": 85},
    {"id": 102, "name": "Priya", "marks": 90},
    {"id": 103, "name": "Amit", "marks": 75}
]

names = [student["name"] for student in students]
print("Student Names:", names)

print("Student Records:")

for student in students:
    print(student)

marks = [student["marks"] for student in students]
print("Marks List:", marks)
```

Output:

```
Student Names: ['Rahul', 'Priya', 'Amit']
Student Records:
{'id': 101, 'name': 'Rahul', 'marks': 85}
{'id': 102, 'name': 'Priya', 'marks': 90}
```

{'id': 103, 'name': 'Amit', 'marks': 75}

Marks List: [85, 90, 75]

6. Read and write data using files.

Program:

```
try:
    name = input("Enter Student Name: ")
    marks = input("Enter Student Marks: ")

    with open("students.txt", "w") as file:
        file.write(name + "," + marks + "\n")

    print("Data written successfully!")

    with open("students.txt", "r") as file:
        data = file.read()

    print("Student Records:")
    print(data)

except OSError:
    print("Error while accessing the file.")
```

Output:

Enter Student Name: Rahul

Enter Student Marks: 85

Data written successfully!

Student Records:

Rahul,85

Expected Output

1. Python collection-based programs.
2. Student/Employee data processing application.